

EXPERIMENT 19

IMPLEMENT CONTINUOUS DEPLOYMENT USING GITHUB ACTIONS TO DEPLOY A DOCKERIZED APPLICATION

Aim

To implement Continuous Deployment using GitHub Actions to automatically build, push, and deploy a Dockerized application.

Requirements

GitHub account, Docker installed, Docker Hub account, a cloud VM/server with Docker installed, and basic Git knowledge.

Procedure

1. Set up GitHub Repository

Create a GitHub repository for the Dockerized application. Initialize the local project with Git, add the application files, commit the changes, and push them to the main branch.

2. Create and Containerize the Application

Create a simple application and a Dockerfile. Build the Docker image using **docker build** and verify it locally by running a container.

3. Create GitHub Actions Workflow

Create a workflow file under **.github/workflows/**. Configure the workflow to trigger whenever new code is pushed to the main branch. The workflow checks out the code, builds the Docker image, pushes it to Docker Hub, and deploys the latest image to the cloud server.

4. Configure Secrets

Add the required Docker Hub and cloud-server credentials as GitHub Actions repository secrets. Sensitive credentials should never be written directly in the workflow file.

5. Test Continuous Deployment

Push a new code change to the main branch and monitor the Actions tab. Verify that the workflow completes successfully, the Docker image is updated, and the application is automatically redeployed.

Screenshot 1 – GitHub Actions Workflow Success

192511064simats-0607 / github-actions-docker

<> Code

🕒 Issues

🔗 Pull requests

🕒 Actions

📁 Projects

📖 Wiki

🛡 Security

📈 Insights

⚙ Settings

Actions

New workflow

All workflows

Docker CI/CD Pipeline

Management

Caches

Runners

Docker CI/CD Pipeline

docker-deploy.yml

Re-run all jobs

⋮

✓ This run has succeeded

main · c3d3e9f

Triggered via push	Status	Total duration	Artifacts
22 minutes ago	Success	2m 37s	—

docker-deploy.yml

on: push

✓ Checkout code2s

→

✓ Build Docker Image27s

→

✓ Push Docker Image18s

↓

✓ Deploy to Server1m 25s

🔄

— +

🔍

Screenshot 2 – Docker Hub Image

The screenshot shows the Docker Hub interface for a repository named 'github-actions-app' under the user '192511064simats-0607'. The page has a dark blue header with the Docker Hub logo, a search bar, and navigation links: 'Explore', 'Repositories', 'Organizations', 'Help', and a profile icon. Below the header, the repository path is shown as '192511064simats-0607 > Repositories > github-actions-app'. A tab bar below the path includes 'General' (selected), 'Tags', 'Builds', 'Collaborators', 'Webhooks', and 'Settings'. The main content area is divided into two columns. The left column shows the repository name with a purple icon, a 'Description' section stating 'This repository does not have a description' with an edit icon, and a clock icon indicating 'Last pushed: 22 minutes ago'. The right column is titled 'Docker commands' and contains the text 'To push a new tag to this repository,' followed by a code block showing the command: `docker push 192511064simats-0607/github-actions-app:latest` with a copy icon. Below these columns is a 'Tags' section stating 'This repository contains 1 tag(s)' with a 'See all' link. A table follows with columns: 'Tag', 'OS', 'Type', 'Pulled', and 'Pushed'. It contains one row for the 'latest' tag, which is marked as a 'Image', has a Linux OS icon, and was pushed '22 minutes ago'.

Tag	OS	Type	Pulled	Pushed
latest	Linux	Image	---	22 minutes ago

Verification

The GitHub Actions workflow shows a successful run with the Docker image build and push stages completed. The Docker Hub repository shows the latest image tag, confirming that the container image was successfully published.

Result

Thus, Continuous Deployment was successfully implemented using GitHub Actions and Docker. The application image was automatically built and pushed after code changes, demonstrating an automated CI/CD deployment workflow.